

從 App Engine Task Queue 發送工作遷移至 Cloud Tasks (模組 8)

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-8-cloudtasks?hl=zh-tw>

步驟數：7

瞭解如何遷移 Python 2 App Engine NDB & 工作佇列 (推送工作) 應用程式至 Cloud NDB (&A)；Cloud Tasks

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 更新設定](#)
5. [5. 修改應用程式程式碼](#)
6. [6. 摘要/清除](#)
7. [7. 其他資源](#)

1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

本程式碼研究室旨在向 Python 2 App Engine 開發人員說明如何從 App Engine 工作佇列 (發送工作) 遷移至 Cloud Tasks。此外，從 App Engine NDB 遷移至 [Cloud NDB](#) 存取 Datastore 時，也會[隱含遷移](#) (主要涵蓋在第 2 模組中)。

我們在第 7 個單元中新增了 **push** 工作的使用方式，並在第 8 個單元中將該用法遷移至 Cloud Tasks，然後在第 9 個單元中繼續使用 Python 3 和 Cloud Datastore。使用工作佇列處理[提取](#)工作的開發人員，將遷移至 Cloud Pub/Sub，並應改為參閱第 18 至 19 節。

未使用工作佇列？

如果您的應用程式未使用 App Engine 工作佇列服務，請略過第 7 到 9 節 (推送工作) 和第 18 到 19 節 (提取工作)，或將這些程式碼研究室當做練習，熟悉工作佇列遷移作業。

在接下來的研究室中

- 以 [Cloud Tasks](#) 取代 [App Engine 工作佇列 \(推送工作\)](#)
- 將 [App Engine NDB](#) 替換為 [Cloud NDB](#) (另請參閱第 2 節)

軟硬體需求

- 擁有[Google Cloud 專案](#)，且已啟用 [GCP 帳單帳戶](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發](#)及[部署](#) App Engine 應用程式的基本知識
- 可正常運作的第 7 模組 App Engine 應用程式 (完成[程式碼研究室](#) [建議] 或從存放區複製[應用程式](#))

問卷調查

2. 背景

[App Engine 工作佇列](#)支援推送和提取工作。為提升應用程式可攜性，Google Cloud 團隊建議您從 Task Queue 等舊版套裝組合服務，遷移至其他 Cloud 獨立或第三方同等服務。

- [工作佇列發送工作](#)使用者應遷移至 [Cloud Tasks](#)。
- [工作佇列提取工作](#)的使用者應遷移至 [Cloud Pub/Sub](#)。

遷移單元 18-19 涵蓋提取工作遷移，單元 7-9 則著重於推送工作遷移。為了從 App Engine 工作佇列發送工作遷移，我們將其用法新增至現有的 Python 2 App Engine 範例應用程式，該應用程式會註冊新的網頁造訪次數，並顯示最近的造訪次數。[第 7 堂程式碼研究室](#)會新增推送工作，刪除最舊的造訪記錄，因為這些記錄不會再顯示，所以不應佔用 Datastore 的額外儲存空間。本第 8 模組程式碼研究室保留相同功能，但將底層佇列機制從工作佇列推送工作遷移至 Cloud Tasks，並重複[第 2 模組](#)的遷移作業，從 App Engine NDB 遷移至 Cloud NDB，以存取 Datastore。

本教學課程包含下列步驟：

1. 設定/準備工作
 2. 更新設定
 3. 修改應用程式程式碼
-

3. 設定/準備工作

本節將說明如何：

1. 設定 Cloud 專案
2. 取得基準範例應用程式
3. (重新) 部署及驗證基準應用程式
4. 啟用新的 Google Cloud 服務/API

這些步驟可確保您從可運作的程式碼開始，並讓範例應用程式準備好遷移至 Cloud 服務。

1. 設定專案

如果您已完成[第 7 堂程式碼研究室](#)，請重複使用該專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案具備可用的帳單帳戶和已啟用的 App Engine 應用程式。找出專案 ID，因為在本程式碼研究室中，每當遇到 `PROJECT_ID` 變數時，您都需要使用該 ID。

2. 取得基準範例應用程式

其中一項必要條件是可正常運作的第 7 模組 App Engine 應用程式：完成[第 7 模組程式碼研究室](#) (建議)，或從存放區複製第 7 模組應用程式。無論您使用自己的程式碼或我們的程式碼，我們都會從第 7 模組的程式碼開始 (「START」)。本程式碼研究室會逐步說明如何遷移，最後會提供類似於第 8 堂課存放區資料夾 ("FINISH") 內容的程式碼。

- 開始：[模組 7 存放區](#)
- 完成：[模組 8 存放區](#)
- [整個存放區](#) (複製或下載 ZIP 檔)

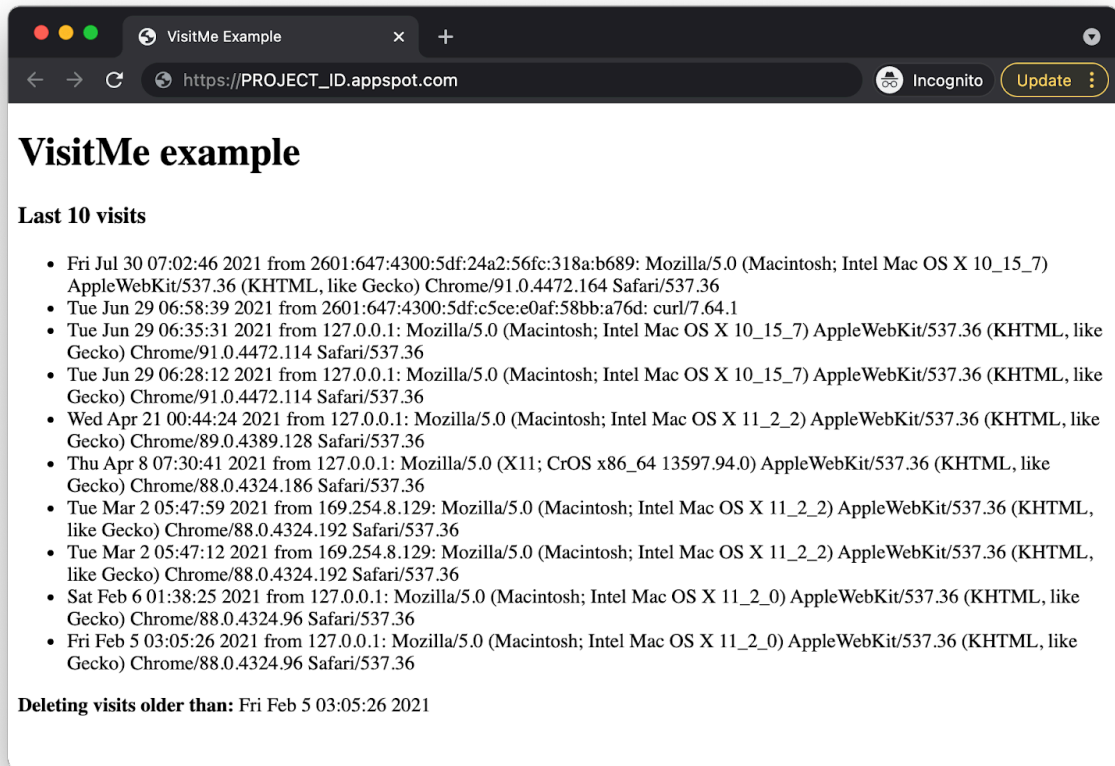
無論您使用哪個第 7 模組應用程式，資料夾都應如下所示，可能也會有 `lib` 資料夾：

```
$ ls
README.md          appengine_config.py  requirements.txt
app.yaml            main.py               templates
```

3. (重新) 部署及驗證基準應用程式

請按照下列步驟部署第 7 堂課的應用程式：

1. 刪除 `lib` 資料夾 (如有)，然後執行 `pip install -t lib -r requirements.txt` 重新填入 `lib`。如果開發電腦同時安裝 Python 2 和 3，可能需要改用 `pip2`。
2. 請確認您已[安裝及初始化](#) `gcloud` 指令列工具，並查看[其用法](#)。
3. (選用) 使用 `gcloud config set project PROJECT_ID` 設定 Cloud 專案，這樣您就不必在發出的每個 `gcloud` 指令中輸入 `PROJECT_ID`。
4. 使用 `gcloud app deploy` 部署範例應用程式
5. 確認應用程式是否正常運作。如果您已完成第 7 模組的程式碼研究室，應用程式會顯示頂尖訪客和最近的造訪記錄 (如下圖所示)。底部會顯示即將刪除的舊工作。

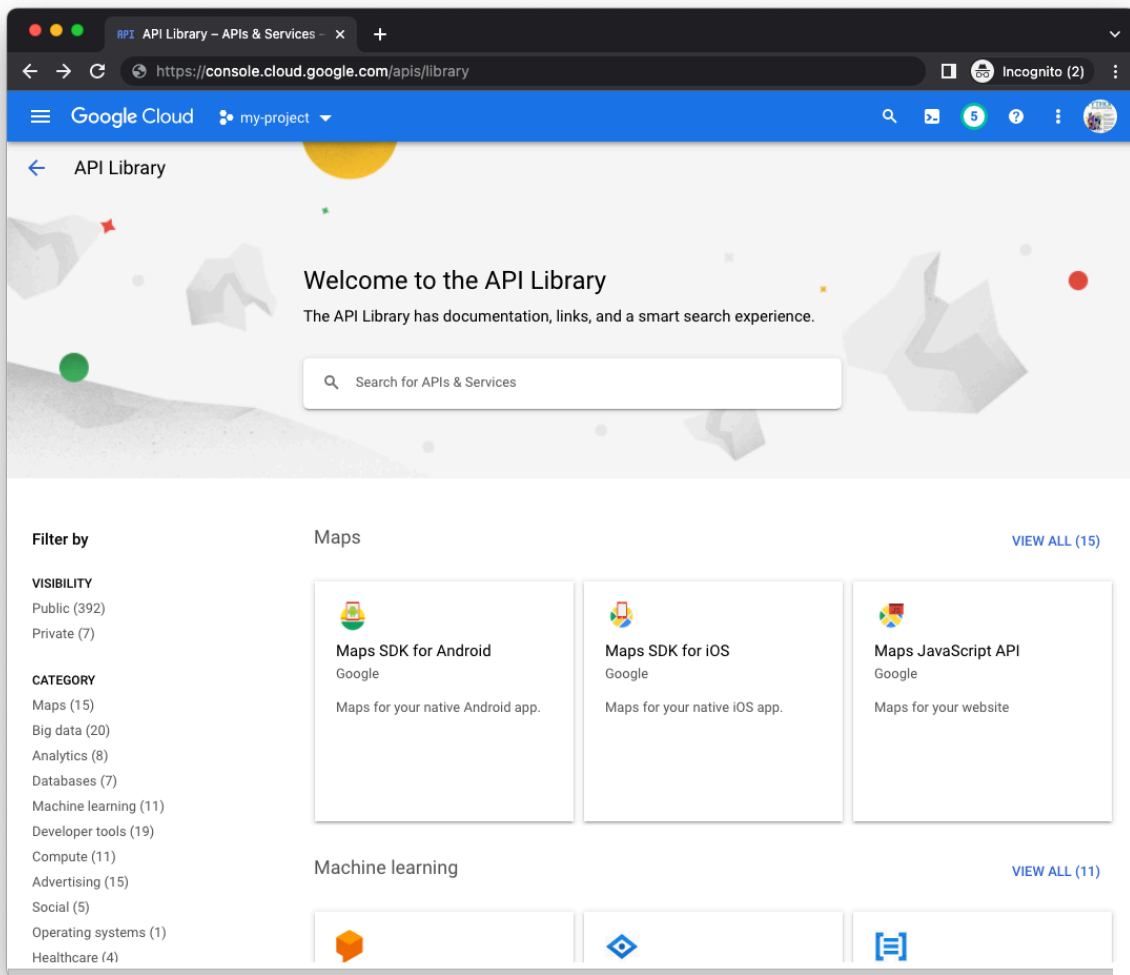


4. 啟用新的 Google Cloud 服務/API

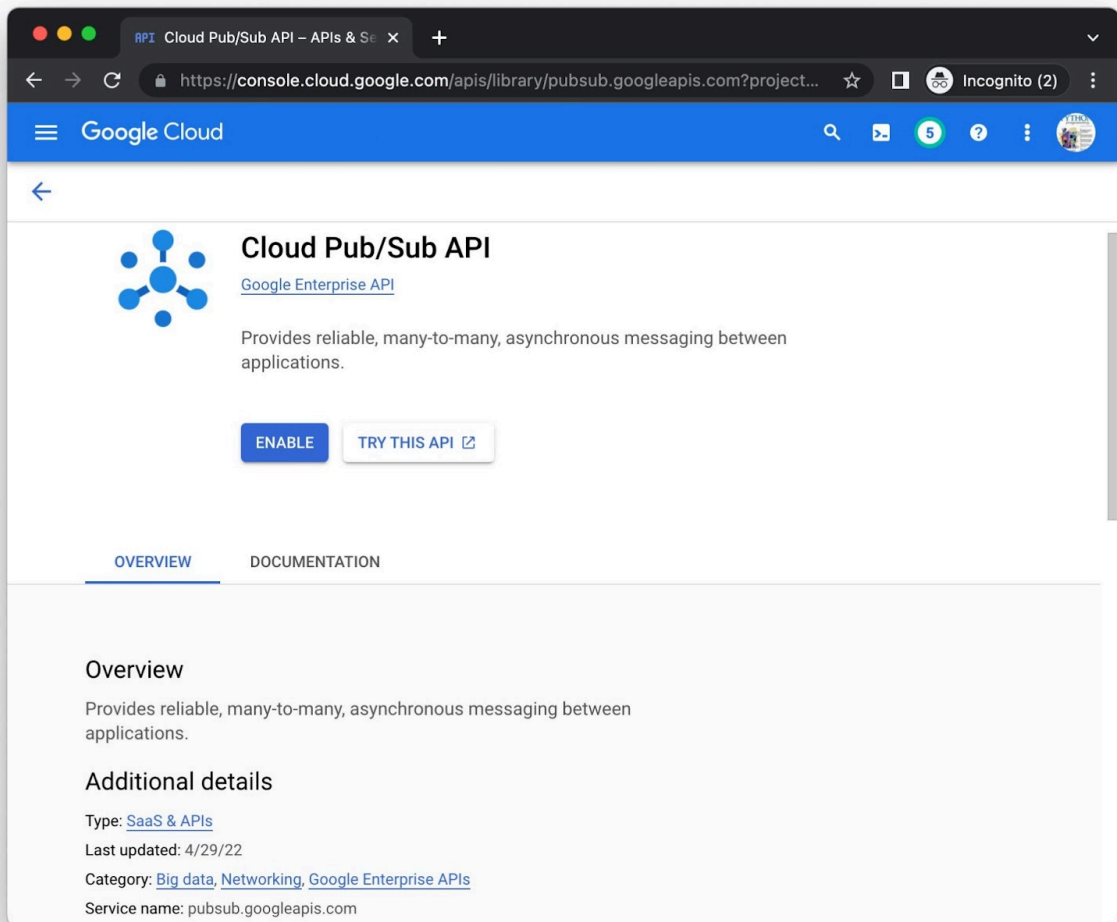
舊版應用程式使用 App Engine 隨附服務，不需要額外設定，但獨立的 Cloud 服務則需要。更新後的應用程式會同時使用 Cloud Tasks 和 [Cloud Datastore](#) (透過 Cloud NDB 用戶端程式庫)。許多 Cloud 產品都有「一律免費」方案配額，包括 [App Engine](#)、[Cloud Datastore](#) 和 [Cloud Tasks](#)。只要用量未超過這些限制，完成本教學課程就不會產生費用。您可以透過 Cloud 控制台或指令列啟用 Cloud API，視個人偏好而定。

透過 Cloud 控制台

在 Cloud 控制台中，前往 [API 管理工具](#) 的「程式庫」頁面 (適用於正確的專案)，然後使用頁面中間的搜尋列，搜尋 Cloud Datastore 和 Cloud Tasks API：



分別點選各個 API 的「啟用」按鈕，系統可能會提示您提供帳單資訊。這是 範例，其中包含 Cloud Pub/Sub API 程式庫頁面 (請勿為這個程式碼研究室啟用 Pub/Sub API，只要啟用 Cloud Tasks 和 Datastore 即可)：



使用指令列

雖然透過控制台啟用 API 具有視覺上的資訊性，但有些人偏好使用指令列。發出 `gcloud services enable cloudtasks.googleapis.com datastore.googleapis.com` 指令，同時啟用這兩個 API：

```
$ gcloud services enable cloudtasks.googleapis.com datastore.googleapis.com
Operation "operations/acat.p2-aaa-bbb-ccc-ddd-eee-ffffff" finished successfully.
```

系統可能會提示您輸入帳單資訊。如要啟用其他 Cloud API 並瞭解其「URI」，請前往各 API 的程式庫頁面底部查看。舉例來說，請觀察上方 Pub/Sub 頁面底部的「服務名稱」`pubsub.googleapis.com`。

完成這些步驟後，專案就能存取 API。現在可以更新應用程式，使用這些 API 了。

4. 更新設定

設定更新是明確因新增 Cloud 用戶端程式庫的使用而發生。無論使用哪一個，如果應用程式未使用任何 Cloud 用戶端程式庫，都必須進行相同的變更。

requirements.txt

第 8 堂課會將第 1 堂課的 App Engine NDB 和 Task Queue 換成 Cloud NDB 和 Cloud Tasks。將 `google-cloud-ndb` 和 `google-cloud-tasks` 都附加至 `requirements.txt`，以便從第 7 堂課加入 `flask`：

```
flask
google-cloud-ndb
google-cloud-tasks
```

這個 `requirements.txt` 檔案沒有任何版本號碼，表示系統會選取最新版本。如有任何不相容問題，請指定版本號碼，鎖定應用程式的工作版本。

app.yaml

使用 Cloud 用戶端程式庫時，Python 2 App Engine 執行階段需要特定第三方套件，即 `grpcio` 和 `setuptools`。Python 2 使用者必須在 `app.yaml` 中列出這類內建程式庫，以及[可用版本或「最新」](#)。如果還沒有 `libraries` 區段，請建立一個，然後加入這兩個程式庫，如下所示：

```
libraries:
- name: grpcio
  version: latest
- name: setuptools
  version: latest
```

遷移應用程式時，應用程式可能已有 `libraries` 區段。如果有的話，且缺少 `grpcio` 和 `setuptools`，請將這兩項新增至現有的 `libraries` 區段。更新後的 `app.yaml` 應如下所示：

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /.
  script: main.app

libraries:
- name: grpcio
  version: latest
- name: setuptools
  version: latest
```

appengine_config.py

`appengine_config.py` 中的 `google.appengine.ext.vendor.add()` 呼叫會將您在 `lib` 中複製 (有時稱為「供應商」或「自行組合」) 的第三方程式庫連結至應用程式。在上述 `app.yaml` 中，我們新增了內建的第三方程式庫，這些程式庫需要 `setuptools.pkg_resources.working_set.add_entry()`，才能將應用程式繫結至 `lib` 中的這些內建套件。以下是原始的模組 1 `appengine_config.py`，以及您更新模組 8 後的模組 1 `appengine_config.py`：

BEFORE:

```
from google.appengine.ext import vendor

# Set PATH to your libraries folder.
PATH = 'lib'
# Add libraries installed in the PATH folder.
vendor.add(PATH)
```

修改後：

```
import pkg_resources
from google.appengine.ext import vendor

# Set PATH to your libraries folder.
PATH = 'lib'
# Add libraries installed in the PATH folder.
vendor.add(PATH)
# Add libraries to pkg_resources working set to find the distribution.
pkg_resources.working_set.add_entry(PATH)
```

您也可以參閱 [App Engine 遷移說明文件](#)，瞭解類似的說明。

5. 修改應用程式程式碼

本節將介紹主要應用程式檔案 `main.py` 的更新內容，說明如何以 Cloud Tasks 取代 App Engine Task Queue 推送佇列。網頁範本和 `templates/index.html` 都不會變更，兩個應用程式的運作方式應完全相同，顯示的資料也一樣。主要應用程式的修改作業可細分為以下四個「待辦事項」：

1. 更新匯入作業和初始化
2. 更新資料模型功能 (Cloud NDB)
3. 遷移至 Cloud Tasks (和 Cloud NDB)
4. 更新 (推送) 工作處理常式

1. 更新匯入作業和初始化

1. 將 App Engine NDB (`google.appengine.ext.ndb`) 和 Task Queue (`google.appengine.api.taskqueue`) 分別換成 Cloud NDB (`google.cloud.ndb`) 和 Cloud Tasks (`google.cloud.tasks`)。
2. Cloud 用戶端程式庫需要初始化及建立「API 用戶端」，並分別指派給 `ds_client` 和 `ts_client`。
3. [工作佇列說明文件](#)指出：「App Engine 提供一個名為 `default` 的預設發送佇列，這個佇列已設定好，隨時可以配合預設設定一起運用。」Cloud Tasks 不提供 `default` 佇列 (因為這是獨立的 Cloud 產品，與 App Engine 無關)，因此需要新程式碼才能建立名為 `default` 的 Cloud Tasks 佇列。
4. App Engine 工作佇列不需要您指定地區，因為它會使用應用程式執行的地區。不過，由於 Cloud Tasks 現在是獨立產品，因此需要區域，且該區域必須與應用程式執行的區域相符。如要建立「完整路徑名稱」做為佇列的專屬 ID，必須提供區域名稱和 Cloud 專案 ID。

上述第三和第四個項目所述的更新，是額外常數和初始化作業的主要內容。請參閱下方的「變更前」和「變更後」部分，並在 `main.py` 頂端進行這些變更。

BEFORE:

```
from datetime import datetime
import logging
import time
from flask import Flask, render_template, request
from google.appengine.api import taskqueue
from google.appengine.ext import ndb

app = Flask(__name__)
```

修改後：

```

from datetime import datetime
import json
import logging
import time

from flask import Flask, render_template, request
from google.cloud import ndb, tasks

app = Flask(__name__)
ds_client = ndb.Client()
ts_client = tasks.CloudTasksClient()

_, PROJECT_ID = google.auth.default()
REGION_ID = 'REGION_ID'      # replace w/your own
QUEUE_NAME = 'default'      # replace w/your own
QUEUE_PATH = ts_client.queue_path(PROJECT_ID, REGION_ID, QUEUE_NAME)

```

2. 更新資料模型功能 (Cloud NDB)

App Engine NDB 和 Cloud NDB 的運作方式幾乎相同。資料模型和 `store_visit()` 函式都沒有重大變更。唯一明顯的差異是，`Visit` 中的 `store_visit()` 實體建立作業現在封裝在 Python `with` 區塊中。Cloud NDB 要求所有 Datastore 存取權都必須在內容管理員中控管，因此需要 `with` 陳述式。以下程式碼片段說明遷移至 Cloud NDB 時的這項微小差異。實作這項變更。

BEFORE:

```

class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor    = ndb.StringProperty()
    timestamp  = ndb.DateTimeProperty(auto_now_add=True)

def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    Visit(visitor='{:}: {}'.format(remote_addr, user_agent)).put()

```

修改後：

```

class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor    = ndb.StringProperty()
    timestamp  = ndb.DateTimeProperty(auto_now_add=True)

def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    with ds_client.context():
        Visit(visitor='{:}: {}'.format(remote_addr, user_agent)).put()

```

3. 遷移至 Cloud Tasks (和 Cloud NDB)

這項遷移作業最重大的變更，是切換底層的佇列基礎架構。這項作業會在 `fetch_visits()` 函式中進行，系統會建立 (推送) 刪除舊版造訪記錄的工作，並將其加入佇列以供執行。不過，第 7 堂課的原始功能仍會保持不變：

1. 查詢最近的造訪記錄。
2. 請勿立即傳回這些造訪記錄，而是儲存最後一個顯示的舊時間戳記 `visit`，這樣就能安全刪除所有較舊的造訪記錄。
3. 使用標準 Python 公用程式，將時間戳記擷取為浮點數和字串，並以各種容量使用這兩者，例如向使用者顯示、新增至記錄檔、傳遞至處理常式等。
4. 以這個時間戳記做為酬載，並以 `/trim` 做為網址，建立推送工作。
5. 最終會透過 HTTP `POST` 呼叫該網址，進而呼叫工作處理常式。

「之前」的程式碼片段說明瞭這項工作流程：

BEFORE:

```
def fetch_visits(limit):
    'get most recent visits & add task to delete older visits'
    data = Visit.query().order(-Visit.timestamp).fetch(limit)
    oldest = time.mktime(data[-1].timestamp.timetuple())
    oldest_str = time.ctime(oldest)
    logging.info('Delete entities older than %s' % oldest_str)
    taskqueue.add(url='/trim', params={'oldest': oldest})
    return data, oldest_str
```

功能維持不變，但 Cloud Tasks 會成為執行平台。為達成這項異動，我們進行了以下更新：

1. 將 `visit` 查詢包裝在 Python `with` 區塊中 (重複執行第 2 模組的 Cloud NDB 遷移作業)
2. 建立 Cloud Tasks 中繼資料，包括預期屬性 (例如時間戳記酬載和網址)，但也要新增 MIME 類型並將酬載編碼為 JSON。
3. 使用 Cloud Tasks API 用戶端建立工作，並提供佇列的中繼資料和完整路徑名稱。

`fetch_visits()` 的變更如下所示：

修改後：

```
def fetch_visits(limit):
    'get most recent visits & add task to delete older visits'
    with ds_client.context():
        data = Visit.query().order(-Visit.timestamp).fetch(limit)
        oldest = time.mktime(data[-1].timestamp.timetuple())
        oldest_str = time.ctime(oldest)
        logging.info('Delete entities older than %s' % oldest_str)
        task = {
            'app_engine_http_request': {
                'relative_uri': '/trim',
                'body': json.dumps({'oldest': oldest}).encode(),
                'headers': {
                    'Content-Type': 'application/json',
                },
            },
        }
        ts_client.create_task(parent=QUEUE_PATH, task=task)
    return data, oldest_str
```

4. 更新 (推送) 工作處理常式

(推送) 工作處理常式函式不需要重大更新，只需要執行即可。這適用於工作佇列或 Cloud Tasks。「程式碼就是程式碼」，他們說。不過，有幾項小變動：

1. 時間戳記酬載會逐字傳遞至工作佇列，但會針對 Cloud Tasks 進行 JSON 編碼，因此抵達時必須進行 JSON 剖析。
2. 使用 Task Queue 對 `/trim` 進行 HTTP POST 呼叫時，隱含的 MIME 類型為 `application/x-www-form-urlencoded`，但使用 Cloud Tasks 時，MIME 類型會明確指定為 `application/json`，因此擷取酬載的方式稍有不同。
3. 使用 Cloud NDB API 用戶端內容管理工具 (第 2 模組：遷移至 Cloud NDB)。

以下是變更工作處理常式 `trim()` 前後的程式碼片段：

BEFORE:

```
@app.route('/trim', methods=['POST'])
def trim():
    '(push) task queue handler to delete oldest visits'
    oldest = request.form.get('oldest', type=float)
    keys = Visit.query(
        Visit.timestamp < datetime.fromtimestamp(oldest)
    ).fetch(keys_only=True)
    nkeys = len(keys)
    if nkeys:
        logging.info('Deleting %d entities: %s' % (
            nkeys, ', '.join(str(k.id()) for k in keys))
        )
        ndb.delete_multi(keys)
    else:
        logging.info('No entities older than: %s' % time.ctime(oldest))
    return '' # need to return SOME string w/200
```

修改後：

```
@app.route('/trim', methods=['POST'])
def trim():
    '(push) task queue handler to delete oldest visits'
    oldest = float(request.get_json().get('oldest'))
    with ds_client.context():
        keys = Visit.query(
            Visit.timestamp < datetime.fromtimestamp(oldest)
        ).fetch(keys_only=True)
        nkeys = len(keys)
        if nkeys:
            logging.info('Deleting %d entities: %s' % (
                nkeys, ', '.join(str(k.id()) for k in keys))
            )
            ndb.delete_multi(keys)
        else:
            logging.info(
                'No entities older than: %s' % time.ctime(oldest))
    return '' # need to return SOME string w/200
```

主要應用程式處理常式 `root()` 和網頁範本 `templates/index.html` 都不會更新。

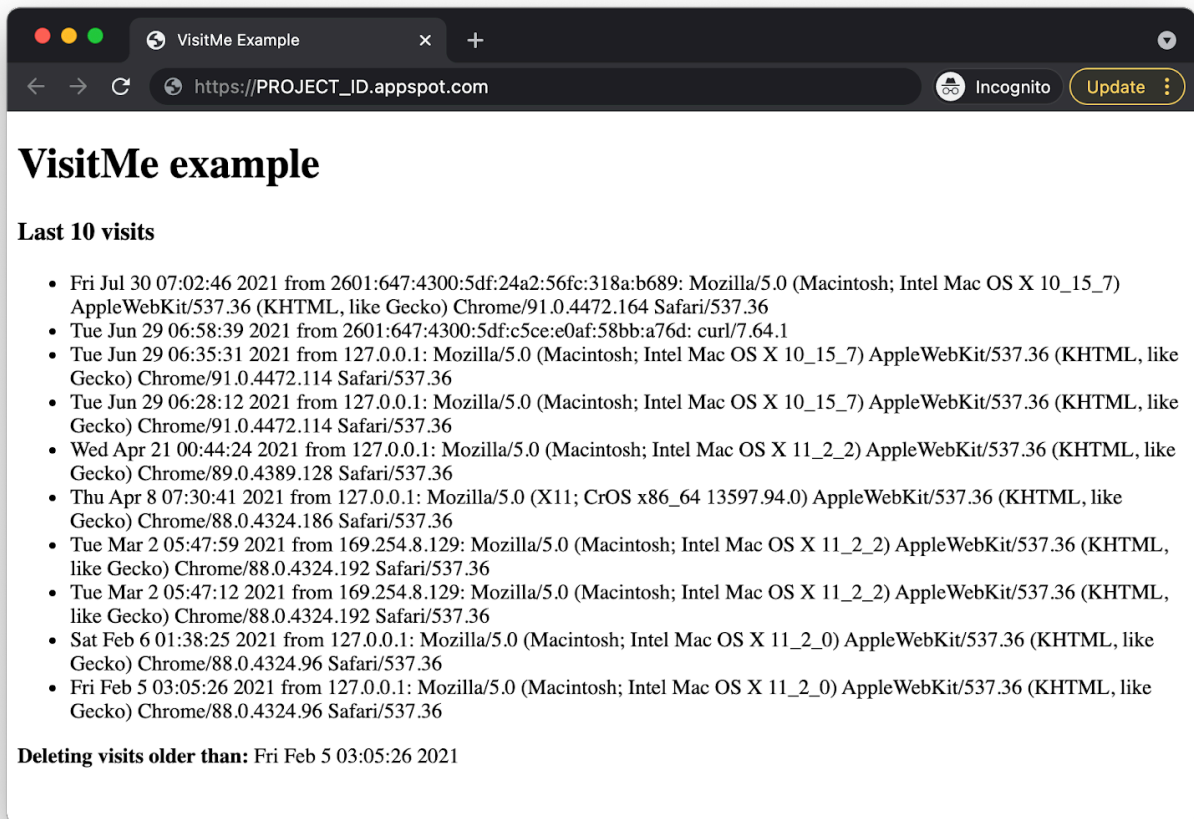
步驟 6 · 約 4 分鐘

6. 摘要/清除

本節將部署應用程式，並驗證應用程式是否正常運作，以及是否反映在輸出內容中，為本程式碼研究室畫下句點。應用程式驗證完成後，請執行任何清理作業，並考慮後續步驟。

部署及驗證應用程式

使用 `gcloud app deploy` 部署應用程式。輸出內容應與第 7 堂課的應用程式相同，但您會發現自己已改用完全不同的推送佇列產品，讓應用程式比以往更具可攜性！



清除所用資源

一般

如果暫時不需要使用，建議[停用 App Engine 應用程式](#)，以免產生費用。不過，如果您想進一步測試或實驗，App Engine 平台提供[免付費配額](#)，只要不超出該用量層級，就不會產生費用。這是指運算費用，但您可能也需要支付相關 App Engine 服務的費用，因此請參閱[其定價頁面](#)瞭解詳情。如果這項遷移作業涉及其他雲端服務，則這些服務會另外計費。無論是哪種情況，請視需要參閱下方的「本程式碼研究室專用」一節。

為求完整揭露，部署至 App Engine 等 Google Cloud 無伺服器運算平台時，會產生[少量建構和儲存空間費用](#)。[Cloud Build](#) 和 [Cloud Storage](#) 都有各自的免費配額。儲存該圖片會耗用部分配額。不過，你所在的區域可能沒有這類免付費層級，因此請留意儲存空間用量，盡量減少潛在費用。您應審查的特定 Cloud Storage「資料夾」包括：

- `console.cloud.google.com/storage/browser/LOC.artifacts.PROJECT_ID.appspot.com/containers/images`

- `console.cloud.google.com/storage/browser/staging.PROJECT_ID.appspot.com`
- 上述儲存空間連結取決於您的 `PROJECT_ID` 和 `*LOCATION`，例如，如果您的應用程式託管於美國，則為「us」。

另一方面，如果您不打算繼續使用這個應用程式或其他相關的遷移 Codelab，並想完全刪除所有內容，請[關閉專案](#)。

本程式碼研究室專用

下列服務是本程式碼研究室的專屬服務。詳情請參閱各項產品的說明文件：

- Cloud Tasks 提供免費方案，詳情請參閱[定價頁面](#)。
- [App Engine Datastore](#) 服務是由 [Cloud Datastore](#) (Cloud Firestore Datastore 模式) 提供，這項服務也有免費方案；詳情請參閱[定價頁面](#)。

後續步驟

至此，我們已完成從 App Engine 工作佇列發送工作遷移至 Cloud Tasks 的作業。如果您有興趣繼續將這個應用程式移植到 Python 3，並從 Cloud NDB 進一步遷移至 Cloud Datastore，請參閱[第 9 個單元](#)。

Cloud NDB 專為 Python 2 App Engine 開發人員而設，提供近乎相同的使用者體驗，但 Cloud Datastore 有自己的原生用戶端程式庫，適用於非 App Engine 使用者或新的 (Python 3) App Engine 使用者。不過，由於 Cloud NDB 適用於 Python 2 和 3，因此您不必遷移至 Cloud Datastore。

Cloud NDB 和 Cloud Datastore 都會存取 Datastore (但方式不同)，因此考慮改用 Cloud Datastore 的唯一理由，是您已使用 Cloud Datastore 建立其他應用程式 (尤其是非 App Engine 應用程式)，並希望採用單一 Datastore 用戶端程式庫。[第 3 模組](#)也會單獨介紹這項選用遷移作業 (不使用 Task Queue 或 Cloud Tasks)。

除了第 3、8 和 9 堂課之外，其他著重於從 App Engine 舊版服務套裝組合遷移的課程還包括：

- [單元 2](#)：從 App Engine NDB 遷移至 Cloud NDB
- [第 12 至 13 個單元](#)：從 App Engine Memcache 遷移至 Cloud Memorystore
- [單元 15 至 16](#)：從 App Engine Blobstore 遷移至 Cloud Storage
- [第 18-19 堂課](#)：從 App Engine 工作佇列 (提取工作) 遷移至 Cloud Pub/Sub

App Engine 不再是 Google Cloud 中唯一的無伺服器平台。如果您有小型 App Engine 應用程式或功能有限的應用程式，並希望將其轉換為獨立的微服務，或是想將單體應用程式拆分成多個可重複使用的元件，這些都是考慮遷移至 [Cloud Functions](#) 的好理由。如果容器化已成為應用程式開發工作流程的一部分，特別是如果工作流程包含 CI/CD (持續整合/持續推送軟體更新或持續部署) 管道，建議遷移至 [Cloud Run](#)。下列單元會說明這些情況：

- 從 App Engine 遷移至 Cloud Functions：請參閱[第 11 堂課](#)
- 從 App Engine 遷移至 Cloud Run：請參閱[第 4 節](#)，瞭解如何使用 Docker 將應用程式容器化；或參閱[第 5 節](#)，瞭解如何在不使用容器、Docker 知識或 `Dockerfile`s 的情況下完成這項作業。

您可以選擇改用其他無伺服器平台，但建議先考量應用程式和用途的最佳選項，再進行任何變更。

無論您接下來要考慮哪個遷移模組，都可以在 [開放原始碼存放區](#) 存取所有 [Serverless Migration Station 內容](#) (程式碼研究室、影片、原始碼 [如有])。此外，該存放區的 `README` 也提供指引，說明要考慮哪些遷移作業，以及遷移模組的相關「順序」。

7. 其他資源

以下列出其他資源，供開發人員進一步瞭解這個或相關的遷移模組，以及相關產品。包括提供內容意見回饋的位置、程式碼連結，以及您可能會覺得實用的各種文件。

程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供第 7 堂課 (START) 和第 8 堂課 (FINISH) 的存放區資料夾連結。

Codelab	Python 2	Python 3
Module 7	code	程式碼 (本教學課程未介紹)
單元 8 (本程式碼研究室)	code	(不適用)

線上資源

以下是可能與本教學課程相關的線上資源：

App Engine 工作佇列和 Cloud Tasks

- [App Engine 工作佇列總覽](#)
- [App Engine 工作佇列推送佇列總覽](#)
- [將 App Engine 工作佇列發送任務遷移至 Cloud Tasks](#)
- [將 App Engine 工作佇列發送任務遷移至 Cloud Tasks 的說明文件範例](#)
- [Cloud Tasks 文件](#)
- [Cloud Tasks Python 用戶端程式庫範例](#)
- [Cloud Tasks 定價資訊](#)

App Engine NDB 和 Cloud NDB (Datastore)

- [App Engine NDB 說明文件](#)
- [App Engine NDB 存放區](#)
- [Google Cloud NDB 說明文件](#)
- [Google Cloud NDB 存放區](#)
- [Cloud Datastore 定價資訊](#)

App Engine 平台

- [App Engine 說明文件](#)

- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [在 Python 2 App Engine 上使用 App Engine 內建程式庫](#)
- [Python 3 App Engine \(標準環境\) 執行階段](#)
- [Python 2 和 3 App Engine \(標準環境\) 執行階段的差異](#)
- [Python 2 到 3 App Engine \(標準環境\) 遷移指南](#)
- [App Engine 定價和配額資訊](#)
- [推出第二代 App Engine 平台 \(2018 年\)](#)
- [比較第一代和第二代平台](#)
- [對舊版執行階段的長期支援](#)
- [說明文件遷移範例](#)
- [社群提供的遷移範例](#)

其他雲端資訊

- [在 Google Cloud Platform 上執行 Python](#)
- [Google Cloud Python 用戶端程式庫](#)
- [Google Cloud「永久免費」方案](#)
- [Google Cloud SDK \(`gcloud` 指令列工具\)](#)
- [所有 Google Cloud 說明文件](#)

影片

- [無伺服器遷移工作站](#)
- [無伺服器 Expeditions](#)
- [訂閱 Google Cloud Tech](#)
- [訂閱 Google Developers](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
